

Amazon ML Challenge 2026

Master Playbook — Business Entity Resolution

Snapshot generated Fri 25 Sep 2026, 03:38 IST | Window: 25 Sep 00:00 IST → 27 Sep 23:59 IST | ~68h20m remaining as of snapshot

1. Status at a Glance

Window	25 Sep 2026, 00:00 IST → 27 Sep 2026, 23:59 IST (~72h)
Team	3–4 eligible students; registration + AWS Builder profile must show complete
Model limit	≤8 billion parameters, MIT or Apache 2.0 license only
Submissions	Max 5 per day, for all 3 days, then the submit button disables
Access	Desktop/laptop only — no mobile. One login per participant — simultaneous logins can terminate your attempt
Leaderboards	Public (live, partial test set) + Private (revealed at the end, decides final rank)
Support	support@unstop.com with a screenshot + registered email; the Google Form linked in your guidelines PDF for hackathon queries

2. The Actual Problem

Entity Resolution: match business records across 3 independent sources that share no common ID. Source 1 is the deduplicated reference set. For every Source 1 entity, find every matching record in Source 2 and Source 3. A match count can be zero (singleton), one, or many.

Fields: entity_id (prefixed S1-/S2-/S3-), business_name, business_address, country. Files are .tsv — always read with sep="\t" explicitly, or you silently get one column.

- **RISK:** Country is an OPEN SET. Training data covers only US and India. The **test set adds France**, with zero training examples. Do not hard-code, filter, or one-hot to {US, India} anywhere in the pipeline — every French test entity still needs a row.

Noise to expect — names: abbreviations (Corp/Corporation, Pvt/Private, Ltd/Limited), DBA/trade names, punctuation (& vs “and”), word-order swaps, typos. Addresses: abbreviations (Rd/Road, St/Street), transliteration variants, missing components, landmark references (“Near SBI ATM”), municipal numbering, reordered components.

3. Output Format — get this exact

Two tab-separated files in output/. **matching_results.tsv is the only one scored** on the leaderboard. candidate_pairs.tsv is not scored but is used to audit your blocking (recall ceiling, reduction ratio) — it is the exact candidate set your matching model ran inference over, the last stage if you block more than once. Every ID in matching_results.tsv must also appear in candidate_pairs.tsv for that entity.

- Exactly one row per Source 1 test entity — missing entities are rejected outright
- Empty matched_entity_ids for singletons; no duplicate IDs within a list; no duplicate source1_entity_id rows

- Matched/candidate IDs must be Source 2 or Source 3 only, and must exist in the test set — no self-matches to Source 1
- Validate locally before every submission: `utils/validate_submission.py` already ships in `student_resource/` (stdlib only) — don't rebuild this, just run it

```
python3 utils/validate_submission.py \  
--matching output/matching_results.tsv \  
--candidate output/candidate_pairs.tsv \  
--test-dir dataset/test
```

4. The Metric — $F_{0.5}$, macro-averaged per entity

$F_{0.5} = (1.25 \times \text{Precision} \times \text{Recall}) / (0.25 \times \text{Precision} + \text{Recall})$, computed **per Source 1 entity, then averaged** across all of them (macro, not a global/micro score).

- Precision is weighted 2× over recall — a false merge (linking two different businesses) costs you more than a missed match. When genuinely unsure, don't match.
- Singletons count fully: predicting an empty list correctly scores 1.0; predicting any match for a true singleton scores 0.0. Getting “no match” right is not free — it's worth as much as any other entity in the macro average.
- Predicting nothing for an entity that does have true matches scores 0.0 (recall 0).

A verified local scorer implementing exactly this (checked against the organizers' own worked example) is in `entity_resolution_metric.py` — see Section 8.

5. Technical Architecture

A. Normalization

- Names: lowercase, strip/standardize legal suffixes (corp/corporation, pvt/private, ltd/limited), normalize punctuation (& vs and), strip extra whitespace
- Addresses: lowercase, standardize abbreviations (rd→road, st→street), tokenize into components where you can (number, street, city) — don't assume every field is populated
- Keep country as a plain string label throughout — never a fixed enum/one-hot

B. Blocking / Candidate Generation — this sets your recall ceiling

Nothing downstream can recover a true match that blocking never surfaces. Cheap, high-recall options, roughly in order of effort:

- Token blocking: candidate if S1 and S2/S3 share any significant name or address token (after stripping stopwords/legal suffixes)
- Sorted-neighborhood: sort by normalized name, compare within a sliding window
- TF-IDF + cosine over a character n-gram vectorizer, threshold or top-k neighbors (sklearn NearestNeighbors with cosine metric scales fine at this size without a GPU)
- MinHash/LSH if brute-force pairwise comparison is too slow once you see actual row counts
- **RISK:** Check blocking recall ceiling immediately with the tool in Section 8 before tuning any matching model — a low ceiling means blocking is the actual bottleneck, not your classifier.
- **RISK:** France has no training examples — keep blocking keys generic (token/n-gram based) rather than anything tuned to US/India address formats specifically, or you'll silently lose France recall.

C. Pairwise Features (per organizers' own tips)

- String similarity: Jaccard on tokens, Levenshtein/edit distance, TF-IDF cosine, longest common substring
- Name-specific: suffix-stripped exact/fuzzy match, acronym match, token overlap ratio
- Address-specific: component-level match where parseable, abbreviation-normalized comparison
- Country match flag — exact equality feature, but never a hard filter
- Optional: a small multilingual sentence-embedding model for semantic/transliteration similarity beyond raw string overlap — tens of millions of params is plenty here, nowhere near the 8B cap

D. Matching Model

Binary classifier on pairwise features — match vs. no-match. LightGBM/XGBoost/CatBoost, same trio as any tabular problem. Positives from ground truth; negatives are the candidate pairs that blocking produced but aren't true matches (hard negatives, since they passed blocking) — expect heavy class imbalance, most candidates are non-matches.

`focal_grad_hess()` in `objectives.py` (built earlier) is a direct fit for this imbalance. Its `find_best_threshold()` currently optimizes plain F1 — swap in entity $F_{0.5}$ -based scoring before using it here, since $F_{0.5}$ picks a different, more conservative threshold than F1 would.

E. Threshold & Post-processing

- Tune the match/no-match probability threshold specifically for macro $F_{0.5}$ on a held-out split, not accuracy or plain F1 — expect the best threshold to sit higher than 0.5, since precision is weighted 2×
- Allow multiple matches per Source 1 entity where the model supports it above threshold — don't force 1:1
- Final assembly: group predictions by `source1_entity_id`, empty list where nothing clears threshold

F. Validation Split

- Split by Source 1 entity, keeping each entity's full match set on one side — splitting individual pairs leaks information across train/val for the same entity
 - Score every candidate change with `entity_resolution_metric.py` locally before burning a submission
 - Trust local macro $F_{0.5}$ over the public leaderboard once the two move together on your first few submissions
-

6. What Gets You Disqualified

- **RISK:** Any external data lookup: entity-resolution APIs/services, government business-registry lookups, geocoding APIs to normalize addresses, any internet-sourced augmentation
 - **RISK:** Proprietary LLM APIs (OpenAI, Anthropic, Google, etc.) — open-source/local only
 - **RISK:** A model over 8B parameters, or without an MIT/Apache 2.0 license
 - **RISK:** Multiple registrations/IDs for the same participant
 - **RISK:** Simultaneous logins across devices — use one laptop per person, don't share logins to “help”
 - **RISK:** Malformed output — missing entities, duplicate IDs, references to non-test IDs — fails validation and isn't evaluated at all, regardless of model quality
-

7. Compute & Environment

This problem is CPU-friendly — string operations and gradient boosting, not heavy deep learning. Don't over-invest in GPU wrangling; it buys less here than it would for an image/VLM problem.

- Check actual downloaded file sizes first (`wc -l`, `ls -lh`). If it's not enormous, skip cloud entirely and run locally on whoever has the most RAM
- Kaggle: free, no card, ~30 GPU-hrs/week if you want it — use if already phone-verified
- Colab: free T4, zero setup beyond a Google login — fastest zero-friction option
- Lightning AI: real persistent terminal (closer to a true CLI workflow than notebook cells) — worth it if someone already knows the platform, not worth learning cold mid-competition
- AWS (if using an existing funded account): best fit for the `run.sh`/EBS/overnight-job architecture, but a brand-new account requesting GPU quota is genuinely unreliable — default G/VT quota is 0 and approval can take minutes to days. Run `aws_bootstrap.sh` (Section 8) to check/request it, but don't block on it — keep Kaggle/Colab as the real fallback

Core stack, install everywhere:

```
pip install polars lightgbm xgboost catboost optuna scikit-learn bitsandbytes transformers wandb
--break-system-packages
```

Run `setup_check.py` (Section 8) on every machine before touching real data — it self-detects the platform and tells you exactly what's missing.

8. Code Already Built & Verified

File	What it does	Verified
<code>entity_resolution_metric.py</code>	Macro $F_{0.5}$ scorer + blocking recall-ceiling diagnostic	Matches organizers' own worked example (0.714) + edge cases

File	What it does	Verified
objectives.py	SMAPE/MAPE/focal-loss objectives for LightGBM/XGBoost, F1 threshold finder	Grad/hess checked vs. finite differences; focal loss exact at gamma=0
setup_check.py	Platform-detecting environment verifier (Kaggle/Colab/AWS/local)	Runs clean, correctly flags missing packages
aws_bootstrap.sh	Checks/requests EC2 GPU quota, sets S3 transfer concurrency	Syntax-checked; needs a live AWS account to run for real

All four are in your outputs from this chat. objectives.py was built before the problem statement was known — its regression losses (SMAPE/MAPE) don't apply to this challenge; keep focal_grad_hess() and find_best_threshold() for the match/no-match classifier.

9. Schedule From This Snapshot (~68h remaining)

Block	Focus	Exit criterion
Now → +6h	Profile actual data scale; naive blocking (token/name overlap) + a trivial rule-based matcher; get ONE valid submission through <code>validate_submission.py</code> and onto the leaderboard	Public LB shows a real score, format confirmed correct
+6h → +18h	Real blocking (TF-IDF/n-gram or LSH); check recall ceiling with the tool in Section 8; build the full pairwise feature set	Blocking recall ceiling above 90% on local val split
+18h → +36h	Train LightGBM/XGBoost/CatBoost matcher on features; tune threshold for local macro F0.5 specifically; confirm local-CV vs public-LB move together	Local F0.5 and public LB track each other on 3 straight submissions
+36h → +54h	Error analysis on worst-scoring entities (false merges first — they cost 2x); verify France/unseen-country generalization specifically; ensemble if time allows	No systematic failure pattern left uninvestigated
+54h → +64h	Freeze the pipeline. Package code/business_entity_resolution/ exactly per Section 11. Re-run end to end on a clean checkout to confirm reproducibility	A stranger could run your README and reproduce both output files
+64h → end	Write the methodology document (<code>Documentation_template.md</code>). Final format re-validation. Submit the zip. Stop touching the model	Package uploaded well before 23:59 IST Day 3

Times are relative to the snapshot above, not the original 00:00 window start — recompute the remaining budget against 27 Sep 23:59 IST as you go.

10. Team Roles

- **Blocking & Data owner** — normalization, candidate generation, recall-ceiling monitoring
 - **Feature owner** — string-similarity + embedding features, feature matrix hygiene
 - **Modeling owner** — classifier training, threshold tuning for $F_{0.5}$, error analysis
 - **Infra & submission owner** — environment setup, `validate_submission.py` on every upload, final zip packaging, methodology doc
-

11. Final Submission Package

```
_submission.zip
output/
  matching_results.tsv
  candidate_pairs.tsv
code/business_entity_resolution/
  src/
  README.md (exact reproduce steps: data -> blocking -> matching -> output)
  requirements.txt (pinned versions)
Documentation_template.md (filled in -- no page limit, prioritize depth)
```

Top 100 teams (by leaderboard + eligibility) get shortlisted and must additionally detail: methodology, candidate-generation/blocking strategy, model architecture and feature engineering, and anything else relevant — the `Documentation_template.md` above should already cover all of this if filled in properly the first time.